

ServiceHub Developer Guide

This document is the source of truth for *how* and *why* ServiceHub is built the way it is. The README tells you what it does and how to run it. This guide tells you how every layer fits together, why each design decision was made, and where the sharp edges are.

If you're new to the codebase, read it top to bottom — every section builds on the previous one.

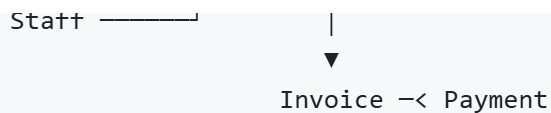
Table of contents

1. [Domain model](#)
 2. [Three-layer architecture](#)
 3. [Request lifecycle](#)
 4. [Database layer](#)
 5. [Repository layer](#)
 6. [Service layer](#)
 7. [GraphQL layer](#)
 8. [Concurrency model](#)
 9. [Money handling](#)
 10. [Scheduling and conflict detection](#)
 11. [Billing lifecycle](#)
 12. [Configuration and environments](#)
 13. [Schema management](#)
 14. [Errors and validation](#)
 15. [Testing strategy](#)
 16. [Deployment notes](#)
 17. [Common gotchas](#)
 18. [Extending the system](#)
-

1. Domain model

ServiceHub revolves around five aggregates plus a single value-style enum module:

```
Client — Appointment — Service
```



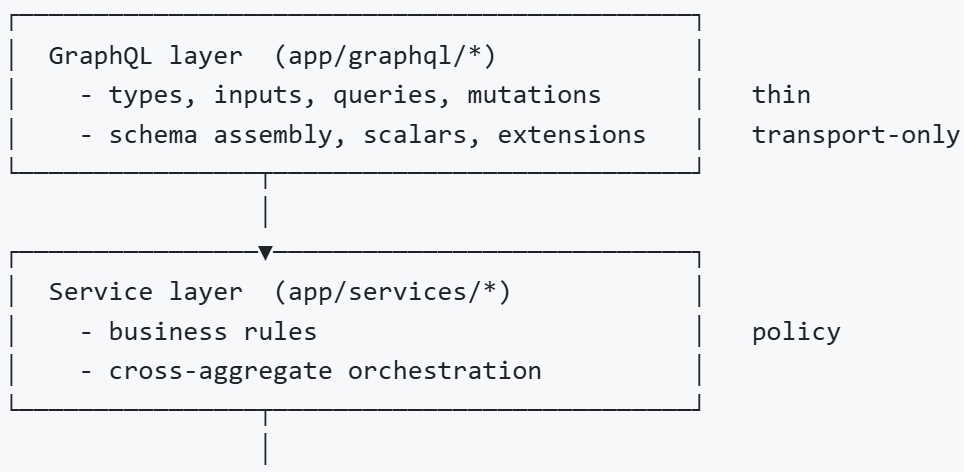
Aggregate	Owns	Identified by
Client	name, email, phone	UUID
Staff	name, email, role, specializations	UUID
Service	name, description, duration, price	UUID
Appointment	client_id, staff_id, service_id,	UUID
	scheduled_at, status, notes	
Invoice	appointment_id (1:1), amount,	UUID
	status, issued_at, paid_at	
Payment	invoice_id (N:1), amount, method,	UUID
	paid_at	

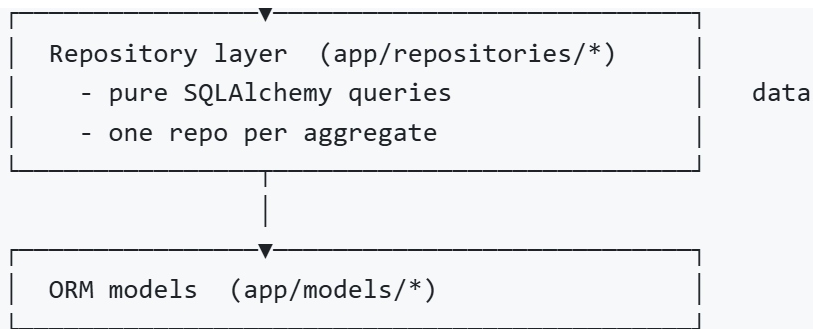
Three Python `enum.Enum` types live in `app/models/enums.py` and are **shared** between the ORM (mapped via `sqlalchemy.Enum`) and the GraphQL schema (wrapped via `strawberry.enum`):

- `AppointmentStatus`: `PENDING` | `CONFIRMED` | `CANCELLED` | `COMPLETED`
- `InvoiceStatus`: `PENDING` | `PAID` | `OVERDUE`
- `PaymentMethod`: `CASH` | `CARD` | `ONLINE`

Sharing the same Python enum prevents drift between the database and the API.

2. Three-layer architecture





Why: if you ever add a REST API, a CLI, or a worker job, only the GraphQL layer needs to change. Repositories and services already encapsulate everything else.

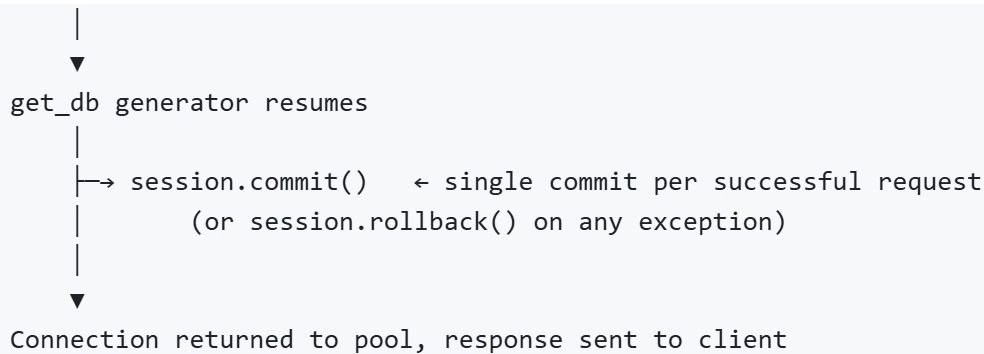
Rule: resolvers should be no more than ~25 lines. They translate GraphQL inputs into service calls and translate the result back. They do **not** contain business logic.

3. Request lifecycle

A typical mutation flows like this:

```

HTTP POST /graphql
  |
  ▼
FastAPI route ← Strawberry GraphQLRouter mounted at /graphql
  |
  ▼
get_context(db = Depends(get_db))
  |               |
  |               └─ AsyncSessionLocal() ← request-scoped session opens
  |
  ▼
Strawberry parses the query, picks the resolver
  |
  ▼
SerializeDatabaseAccess.resolve() ← acquires per-request asyncio.Lock
  |
  ▼
Resolver code (mutation function)
  |
  └─ AppointmentService(db).create(...)
    |
    └─ AppointmentRepository(db).find_staff_conflicts(...)
      |
      └─ AppointmentRepository(db).add(appointment)
        |
        └─ session.flush() ← writes, no commit yet
  |
  ▼
Resolver returns AppointmentType
  |
  ▼
Strawberry serializes response
  
```



The two non-obvious pieces are:

- **Single commit point.** The session is committed exactly once per request, in `get_db`, after the resolver returns. Repositories only `flush()`. This guarantees that a multi-step mutation rolls back cleanly if any step fails.
- **Per-request lock.** The lock in `Context` lives for one request and is acquired around every resolver call. See §8 for why.

4. Database layer

`app/database.py` defines the engine and session factory.

```
engine = create_async_engine(
    settings.database_url,
    pool_size=settings.database_pool_size,
    max_overflow=settings.database_max_overflow,
    pool_timeout=settings.database_pool_timeout,
    pool_pre_ping=True,
    echo=settings.debug,
)

AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
)
```

Key flags

Flag	Why it matters
<code>pool_pre_ping=True</code>	Supabase aggressively drops idle connections. Without pre-ping, the first query on a stale socket fails with a <code>ConnectionResetError</code> .
	Mandatory for async sessions. Otherwise touching any attribute

<code>expire_on_commit=False</code>	after commit triggers a sync lazy-load and raises <code>MissingGreenlet</code> .
<code>echo=settings.debug</code>	Logs every SQL statement when <code>DEBUG=True</code> .

get_db dependency

```

async def get_db() -> AsyncGenerator[AsyncSession, None]:
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise

```

This is the **only** place commits happen. Repositories never commit, which is what gives mutations atomic semantics.

5. Repository layer

One repository per aggregate, plus `BillingRepository` which owns both invoices and payments because they share a transactional boundary.

BaseRepository[ModelType]

A small generic base provides `get_by_id` , `list_all` , `add` , `delete` . Subclasses bind `model = ...` and add domain-specific queries.

```

class BaseRepository(Generic[ModelType]):
    model: type[ModelType]

    def __init__(self, db: AsyncSession):
        self.db = db

    async def add(self, obj):
        self.db.add(obj)
        await self.db.flush()
        await self.db.refresh(obj)
        return obj

```

Why `flush()` instead of `commit()`?

`flush()` writes pending changes to the DB inside the current transaction. The transaction

itself is committed by `get_db`. This means a mutation can do many writes and roll back atomically if any step fails — even one in a different repository.

Domain-specific queries

Repository	Notable methods
<code>ClientRepository</code>	<code>get_by_email</code>
<code>StaffRepository</code>	<code>get_by_email</code>
<code>ServiceRepository</code>	<code>list_active</code>
<code>AppointmentRepository</code>	<code>list_filtered(status, client_id, ...)</code> , <code>find_staff_conflicts(...)</code> , <code>list_for_client</code> , <code>list_for_staff</code>
<code>BillingRepository</code>	<code>list_invoices</code> , <code>get_invoice_by_appointment_id</code> , <code>get_payments_for_invoice</code> , <code>sum_payments_for_invoice</code> , <code>add_payment</code>

6. Service layer

Two service classes, one per business workflow:

AppointmentService (`app/services/appointment.py`)

Owns booking rules:

- Validates that the referenced client, staff, and service exist
- Rejects bookings against an inactive service
- Performs duration-aware overlap detection (see §10)
- Mediates status transitions

Custom exceptions: `AppointmentConflictError`, `AppointmentNotFoundError`.

BillingService (`app/services/billing.py`)

Owns invoice and payment lifecycle:

- `generate_invoice_for_appointment(appt_id, override_amount=None)` — derives the amount from the service price by default; refuses if an invoice already exists.
- `record_payment(invoice_id, amount, method)` — inserts the payment, then recomputes invoice status.
- `_recompute_invoice_status(invoice)` — sums all payments; if total $\geq$ invoice amount and status isn't already PAID, transitions to PAID and stamps `paid_at`.

- `mark_overdue(invoice_id)` — only transitions PENDING invoices.

Custom exceptions: `InvoiceAlreadyExistsError`, `InvoiceNotFoundError`, `AppointmentNotFoundError`.

Why services exist at all: GraphQL resolvers stay thin and trivially testable, and the same business rules are reusable from any future surface (CLI, scheduled jobs, REST).

7. GraphQL layer

Structure

```
graphql/
├── scalars.py          Decimal scalar
├── context.py          Context class (db + db_lock) + get_context
├── extensions.py        SerializeDatabaseAccess
├── schema.py           strawberry.Schema(...)
├── types/              output types (one file per domain)
│   ├── enums.py        wraps Python enums via strawberry.enum
│   ├── client.py        ClientType
│   ├── staff.py         StaffType
│   ├── service.py       ServiceType
│   ├── appointment.py   AppointmentType (nested resolvers!)
│   ├── invoice.py       InvoiceType (with amountPaid computed field)
│   └── payment.py       PaymentType
├── inputs/             input types (Create + Update per domain)
├── queries/            one mixin per domain, composed into Query
└── mutations/          one mixin per domain, composed into Mutation
```

Composition pattern

The root `Query` and `Mutation` types are assembled by multiple inheritance from per-domain mixins:

```
@strawberry.type
class Query(
    ClientQueries,
    StaffQueries,
    ServiceQueries,
    AppointmentQueries,
    BillingQueries,
):
    pass
```

Adding a new domain means writing one queries mixin, one mutations mixin, and adding two lines to these compositors. No central file balloons.

two lines to these composers. No central fire balloons.

Output types and `from_orm`

Every type has a `from_orm(obj)` classmethod that takes the SQLAlchemy model instance and returns the GraphQL type. This is intentionally explicit (no auto-conversion magic) so adding a derived field doesn't silently break the API contract.

```
@classmethod
def from_orm(cls, obj) -> "ClientType":
    return cls(
        id=strawberry.ID(str(obj.id)),
        name=obj.name,
        email=obj.email,
        phone=obj.phone,
        created_at=obj.created_at,
        updated_at=obj.updated_at,
    )
```

Nested resolvers

`AppointmentType.client`, `.staff_member`, `.service`, `.invoice` are declared as `@strawberry.field` async methods. Each does its own DB fetch using `info.context.db`. This is intentionally simple — see §17 for the N+1 caveat and how to upgrade to DataLoaders later.

Strawberry's lazy type references

To avoid circular imports between e.g. `AppointmentType` and `ClientType`, nested-resolver return types use:

```
from typing import TYPE_CHECKING, Annotated
import strawberry

if TYPE_CHECKING:
    from app.graphql.types.client import ClientType

@strawberry.field
async def client(self, info) -> Annotated[
    "ClientType", strawberry.lazy("app.graphql.types.client")
]:
    ...
```

`strawberry.lazy(...)` defers the import until schema build time.

Inputs and `strawberry.UNSET`

`Update*Input` fields default to `strawberry.UNSET` (not `None`) so the mutation can distinguish *omitted* (don't change) from *explicitly null* (clear the value). Resolvers check via `value is not strawberry.UNSET`.

Custom scalars

`Decimal` is the only custom scalar:

```
Decimal = strawberry.scalar(  
    NewType("Decimal", PyDecimal),  
    serialize=lambda v: str(v),  
    parse_value=lambda v: PyDecimal(str(v)),  
)
```

It's serialized as a **string** because JSON numbers are double-precision floats in most clients, which corrupts currency.

`DateTime` uses Strawberry's built-in scalar (ISO 8601).

8. Concurrency model

This is the trickiest part of the codebase, so it gets its own section.

The problem

Strawberry (via `graphql-core`) executes sibling fields concurrently using `asyncio.gather`. So a query like:

```
{  
  clients { id }  
  staff   { id }  
  services { id }  
}
```

...fires three resolvers in parallel. They all share the **same** request-scoped `AsyncSession` (because `get_context` only opens one session per request). SQLAlchemy's async session is **not** safe for concurrent use:

```
sqlalchemy.exc.InvalidRequestError:  
This session is provisioning a new connection;  
concurrent operations are not permitted
```

The fix: `SerializeDatabaseAccess` extension

`app/graphql/extensions.py` defines:

```
class SerializeDatabaseAccess(SchemaExtension):
    async def resolve(self, _next, root, info, *args, **kwargs):
        async with info.context.db_lock:
            result = _next(root, info, *args, **kwargs)
            if inspect.isawaitable(result):
                return await result
            return result
```

The lock is created in `Context.__init__`, so each request gets its own. Resolvers within one request run serially; across requests they're fully concurrent (each request has its own session and its own lock).

Why not per-resolver sessions?

Multi-step mutations like `completeAppointment` do two writes (update appointment + create invoice) that must roll back atomically on any failure. Splitting into per-resolver sessions would break atomicity.

Cost

Inside one request, you lose intra-request parallelism — but a single DB connection cannot serve parallel queries anyway, so this isn't real parallelism, just bookkeeping overhead.

9. Money handling

Currency is **never** stored or computed as a float. Anywhere money appears:

Layer	Type
PostgreSQL	<code>NUMERIC(10, 2)</code>
SQLAlchemy	<code>Numeric(10, 2)</code> → Python <code>Decimal</code>
Service	<code>decimal.Decimal</code>
GraphQL	custom <code>Decimal</code> scalar (string)
JSON wire	string, e.g. <code>"45.00"</code>

Doing `Decimal("0.1") + Decimal("0.2")` gives `Decimal("0.3")`. Doing `0.1 + 0.2` gives `0.30000000000000004` and your invoices stop matching.

When clients send amounts in mutations, they send strings (`"45.00"`), which Strawberry parses into `Decimal` via the `parse_value` hook on the scalar.

parses into `decimal` via the `parse_value` hook on the scalar.

10. Scheduling and conflict detection

Two layers of protection against double-booking:

1. Database-level uniqueness (hard)

```
__table_args__ = (
    UniqueConstraint("staff_id", "scheduled_at", name="uq_staff_scheduled_at"),
)
```

This catches the race condition where two clients book the same staff member at the same `scheduled_at` simultaneously.

2. Service-level overlap detection (smart)

The DB constraint only catches *exact* time matches. To catch overlapping bookings (e.g. a 60-minute service starting at 14:00 vs a 30-minute service starting at 14:30), the service layer queries:

```
SELECT a.* FROM appointments a
JOIN services s ON a.service_id = s.id
WHERE a.staff_id = :staff_id
      AND a.status != 'CANCELLED'
      AND a.scheduled_at < :requested_end
      AND a.scheduled_at + make_interval(mins => s.duration_minutes) > :requested_start
```

This is the standard "interval overlap" predicate: `existing.start < requested.end AND existing.end > requested.start`.

`make_interval(mins => ...)` is a PostgreSQL function that builds the interval at query time using each appointment's own service duration — so the overlap math always uses the correct duration even if services have different lengths.

CANCELLED appointments are excluded so the slot can be re-booked.

When updating an existing appointment, `exclude_appointment_id` skips the row being updated so it doesn't conflict with itself.

11. Billing lifecycle

```

Appointment.status -> COMPLETED
|
▼
BillingService.generate_invoice_for_appointment(appt_id)
|
├─ amount = service.price (or override)
|
▼
Invoice(status=PENDING, issued_at=now())
|
▼
recordPayment mutation called repeatedly
|
▼
Payment row inserted (any amount, any method)
|
▼
BillingService._recompute_invoice_status(invoice)
|
├─ total_paid = SUM(payments.amount)
|
├─ if total_paid >= invoice.amount AND status != PAID:
|     invoice.status = PAID
|     invoice.paid_at = now()
|
|
▼
(or, separately, mark_invoice_overdue when SLA breached)

```

Partial payments

Invoice.amount = \$200, Payment(100, CASH) then Payment(100, CARD) → status flips to PAID after the second payment.

Overpayment

Invoice.amount = \$200, Payment(250, ...) → status PAID, the overpayment is recorded as-is. The system does not auto-refund or generate credit memos (out of scope).

Idempotent completion

completeAppointment calls generate_invoice_for_appointment. If an invoice already exists, InvoiceAlreadyExistsError is caught silently so the mutation is idempotent.

markInvoiceOverdue

Only transitions invoices in PENDING status. PAID invoices stay PAID. Useful when a scheduled job sweeps invoices past their due date.

12. Configuration and environments

`app/config.py` uses `pydantic-settings` to load from `.env` :

```
class Settings(BaseSettings):
    database_url: str
    database_pool_size: int = 5
    database_max_overflow: int = 10
    database_pool_timeout: int = 30
    debug: bool = False
    allowed_origins: list[str] = ["http://localhost:3000"]
```

Variable	Notes
<code>DATABASE_URL</code>	Must use <code>postgresql+asyncpg://</code> scheme. URL-encode <code>@</code> → <code>%40</code> etc. in passwords.
<code>DATABASE_POOL_SIZE</code>	Set to <code>1</code> when using PgBouncer (Supabase port 6543).
<code>ALLOWED_ORIGINS</code>	JSON array string, e.g. <code>["http://localhost:3000"]</code> .
<code>DEBUG</code>	Enables SQL echo + DEBUG-level logs.

Supabase connection notes

Path	Host	Port	Pool size	When
Direct	<code>db.<ref>.supabase.co</code>	5432	5–10	dev, IPv6 networks
PgBouncer	<code>aws-0- <region>.pooler.supabase.com</code>	6543	1	production

The direct endpoint is IPv6-only on Supabase's free tier. If your network is IPv4-only and DNS times out, switch to the pooler URL.

13. Schema management

Two paths to the same schema:

Option A — `schema.sql` (recommended for quick setup)

A standalone, idempotent DDL file at the repo root. Paste into the Supabase SQL editor. Creates enums, tables, indexes, FKs, and `updated_at` triggers (via a `set_updated_at()` PL/pgSQL function)

FL/pgSQL function).

Option B — Alembic

`alembic.ini` + `app/migrations/env.py` (async-aware). The `env.py` loads the URL from `Settings` rather than `alembic.ini` so env vars remain the single source of truth.

```
alembic revision --autogenerate -m "describe_change"
alembic upgrade head
```

Caveat: `configparser` (Alembic's INI backend) treats `%` as interpolation syntax. If your URL contains URL-encoded characters (e.g. `%40` for an `@` in the password), you may need to escape as `%%40` when calling `set_main_option`. The standalone `schema.sql` sidesteps this entirely.

Trigger vs. ORM `onupdate`

The ORM's `onupdate=func.now()` runs on **the application's** UPDATE statements only. The SQL file installs a DB-level `BEFORE UPDATE` trigger so `updated_at` stays correct even for direct SQL edits in the Supabase dashboard. Both are present and compatible.

14. Errors and validation

Currently the resolvers raise plain `Exception` with a descriptive message, which Strawberry surfaces in the GraphQL `errors` array:

```
try:
    obj = await svc.create(...)
except AppointmentConflictError as exc:
    raise Exception(f"Conflict: {exc}") from exc
```

This is intentionally minimal — for a production deployment you should:

- Define typed error types and `strawberry.union` result types so clients can branch on error variants
- Add input validation via Pydantic models or custom validators (e.g. `email-validator` is already installed for that)
- Map known DB errors (`UniqueViolationError` on `uq_staff_scheduled_at` , FK violations, etc.) to user-friendly errors

See §18 for upgrade guidance.

15. Testing strategy

Tests aren't checked in yet, but the architecture is built to make them straightforward:

Layer	What to test	Tool
Repositories	Each query against a real Postgres test DB	pytest-asyncio + a scratch Supabase project or a docker-compose Postgres
Services	Conflict detection, billing transitions	pytest with mock repos OR real DB
GraphQL	End-to-end resolver behavior	<code>httpx.AsyncClient</code> against the FastAPI app

Suggested layout

```
tests/
├─ conftest.py           # async DB fixture, FastAPI test client
├─ test_repositories.py
├─ test_appointment_service.py
├─ test_billing_service.py
└─ test_graphql.py
```

Useful fixture sketch

```
@pytest.fixture
async def db_session():
    async with AsyncSessionLocal() as session:
        async with session.begin():
            yield session
        await session.rollback() # never commit in tests
```

`pytest`, `pytest-asyncio`, and `httpx` are already in `requirements.txt`.

16. Deployment notes

Server

```
uvicorn app.main:app \
  --host 0.0.0.0 \
  --port 8000 \
  --workers 4 \
  --proxy-headers
```


`--workers > 1` only works when each worker has its own DB pool, which is the default. Together with PgBouncer (Supabase port 6543), set `DATABASE_POOL_SIZE=1` per worker to avoid connection storms.

Production checklist

- `DEBUG=false` in env
- `DATABASE_URL` points to PgBouncer (port 6543), `DATABASE_POOL_SIZE=1`
- `ALLOWED_ORIGINS` restricted to actual frontend hosts
- Run behind HTTPS (uvicorn + reverse proxy or `--ssl-keyfile`)
- Real auth in front of `/graphql` (see §18)
- Structured logging shipped to a sink (Datadog, Loki, etc.)
- Health endpoint wired to the orchestrator's liveness probe

Lifespan handler

`app/main.py` registers an async lifespan that disposes the engine on shutdown:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    yield
    await engine.dispose()
```

This closes pooled connections cleanly on SIGTERM.

17. Common gotchas

Concurrent session error

This session is provisioning a new connection; concurrent operations are not permitted

If you see this, the `SerializeDatabaseAccess` extension was disabled or a resolver bypassed `info.context.db_lock`. Make sure the extension is in `schema.py`'s `extensions=[...]` list.

MissingGreenlet on attribute access

MissingGreenlet: greenlet_spawn has not been called

You touched a lazy-loaded relationship after commit. Either set `expire_on_commit=False` (already done) or eager-load the relationship with `selectinload(...)` in the query.

Alembic complains about % in URL

```
ValueError: invalid interpolation syntax
```

`configparser` treats `%` as interpolation. Escape as `%%` in `set_main_option`, or use `schema.sql` instead.

asyncpg loaded as psycopg2

```
The asyncio extension requires an async driver to be used. The loaded 'psycopg2' is not async.
```

Your `DATABASE_URL` is missing the `+asyncpg` driver tag. Fix the scheme to `postgresql+asyncpg://...`.

N+1 on nested resolvers

`AppointmentType.client` / `.staff_member` /etc. do one query each. A list of 20 appointments + their clients = 21 queries. Acceptable for now; for production-scale lists, add per-request `DataLoaders` (see §18).

Square brackets in Supabase password

The Supabase docs use `[YOUR-PASSWORD]` as a placeholder. Don't paste those brackets into `.env` literally. URL-encode special chars in the actual password (e.g. `@` → `%40`, `#` → `%23`).

18. Extending the system

Add a new domain

1. ORM model in `app/models/foo.py` → register in `app/models/__init__.py`
2. Migration (Alembic) or DDL appended to `schema.sql`
3. Repository in `app/repositories/foo.py`
4. (Optional) Service in `app/services/foo.py`
5. Output type in `app/graphql/types/foo.py`
6. Inputs in `app/graphql/inputs/foo.py`
7. Queries mixin in `app/graphql/queries/foo.py`
8. Mutations mixin in `app/graphql/mutations/foo.py`
9. Wire mixins into `Query` / `Mutation` in `app/graphql/queries/__init__.py` and `app/graphql/mutations/__init__.py`

Add authentication

Recommended: JWT-bearer auth as a FastAPI dependency, with the user claims threaded into `Context` :

```
class Context(BaseContext):
    def __init__(self, db, user=None):
        super().__init__()
        self.db = db
        self.db_lock = asyncio.Lock()
        self.user = user

async def get_context(
    db: AsyncSession = Depends(get_db),
    user = Depends(get_current_user), # your JWT verifier
) -> Context:
    return Context(db=db, user=user)
```

Resolvers can then check `info.context.user` and raise `Unauthorized` when missing.

Add DataLoaders (fix N+1)

Strawberry ships `strawberry.dataloader.DataLoader` . Create one per domain, attach to `Context` in `__init__` , and use it from nested resolvers:

```
async def load_clients(ids: list[uuid.UUID]) -> list[Client]:
    result = await session.execute(
        select(Client).where(Client.id.in_(ids))
    )
    by_id = {c.id: c for c in result.scalars()}
    return [by_id.get(i) for i in ids]
```

Add cursor-based pagination

The current resolvers use `skip / limit` . For large lists (`appointments` especially), implement Relay-style connections: `AppointmentConnection { edges { node cursor }, pageInfo, totalCount }` . Encode cursors as `base64(scheduled_at|UUID)` .

Add typed error unions

Replace `raise Exception("...")` with:

```
@strawberry.type
class NotFoundError:
    message: str

ClientResult = strawberry.union("ClientResult", [ClientType, NotFoundError])
```

Mutations return `ClientResult` so clients can branch on `__typename`.

Add subscriptions (real-time)

Strawberry supports GraphQL subscriptions via WebSockets. Useful for "new appointment booked" or "invoice paid" notifications. Will need Postgres LISTEN/NOTIFY or an external pub-sub since PgBouncer in transaction mode doesn't support `LISTEN` — fall back to direct port 5432 for the subscription path, or use Supabase Realtime.

Appendix A — File index

File	Purpose
<code>app/main.py</code>	FastAPI app, CORS, GraphQL mount, health
<code>app/config.py</code>	Pydantic settings
<code>app/database.py</code>	Async engine, sessionmaker, get_db
<code>app/models/base.py</code>	DeclarativeBase + TimestampMixin
<code>app/models/enums.py</code>	Shared status enums
<code>app/models/{client,staff,service,appointment,invoice,payment}.py</code>	ORM models
<code>app/repositories/base.py</code>	Generic CRUD
<code>app/repositories/{client,staff,service,appointment,billing}.py</code>	Per-aggregate queries
<code>app/services/{appointment,billing}.py</code>	Business logic
<code>app/graphql/scalars.py</code>	Decimal scalar
<code>app/graphql/context.py</code>	Context (db + db_lock + get_context)
<code>app/graphql/extensions.py</code>	SerializeDatabaseAccess
<code>app/graphql/schema.py</code>	Root schema assembly
<code>app/graphql/types/*.py</code>	GraphQL output types
<code>app/graphql/inputs/*.py</code>	GraphQL input types

<code>app/graphql/queries/*.py</code>	Query mixins
<code>app/graphql/mutations/*.py</code>	Mutation mixins
<code>schema.sql</code>	Standalone DDL for Supabase SQL editor
<code>alembic.ini</code> , <code>app/migrations/</code>	Alembic config + asyn env

Appendix B — Glossary

- **Aggregate** — a cluster of related entities treated as one unit. Here: Client, Staff, Service, Appointment, Invoice, Payment.
- **Repository** — a class that issues SQL for one aggregate.
- **Service** — a class that orchestrates repositories to enforce business rules.
- **Resolver** — a Strawberry function that produces the value for a GraphQL field.
- **Context** — the per-request object Strawberry passes to every resolver. Holds the DB session and concurrency lock.
- **Aggregate root** — the entity through which the rest of the aggregate is accessed (e.g. Invoice for Payments)